

LAPORAN PRAKTIKUM
SISTEM ADMINISTRASI DAN INFORMASI TERDISTRIBUSI
PERTEMUAN 11
ORKESTRASI DOCKER DENGAN DOCKER COMPOSE



Disusun oleh:

Nama : Hanan Fijananto
NIM : 24/538946/SV/24555
Kelas : B2
Dosen Pengampu : Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT
LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

A Tujuan

1. Mahasiswa memahami perbedaan antara Dockerfile (membangun image) dan Docker Compose (menjalankan container).
2. Mahasiswa mampu menstranslasikan perintah CLI docker run yang panjang menjadi file konfigurasi docker-compose.yml
3. Mahasiswa mampu mengimplementasikan fitur Volumes untuk Hot-Reloading (perubahan kode langsung terlihat tanpa rebuild).
4. Mahasiswa mampu mengelola environment variables menggunakan .env yang terintegrasi dengan Docker Compose.

B Dasar Teori

B.1 Docker Compose

Docker-Compose adalah alat untuk mendefinisikan dan menjalankan satu atau beberapa container yang saling terkait dengan sebuah command. Pada implementasinya kita dapat menggunakannya dengan membuat sebuah file berekstensi `yml/yaml` yang di dalamnya terdapat konfigurasi-konfigurasi terhadap service aplikasi yang akan dijalankan. Gambaran sederhananya seperti menyatukan semua Dockerfile dari setiap service aplikasi ke dalam sebuah file `yml` (docker-compose file), dan selanjutnya dengan sebuah command, pengembang dapat membuat dan men-start semua service yang telah didefinisikan pada file `yml` tersebut. Script konfigurasinya juga sangat mudah dan mirip dengan argumen-argumen saat kita ingin menjalankan sebuah Dockerfile. Kita bisa menggunakannya di semua lingkungan kerja: production, staging, development, dan testing [1].

B.2 Orkestrasi Container

Kontainer adalah komponen aplikasi ringan yang dapat di eksekusi yang menggabungkan kode sumber aplikasi dengan semua pustaka sistem operasi (OS) dan ketergantungan yang diperlukan untuk menjalankan kode di lingkungan apa pun.

Kemampuan untuk membuat kontainer sudah ada sejak beberapa dekade yang lalu, tetapi baru tersedia secara luas pada tahun 2008 ketika Linux menyertakan fungsi kontainer di dalam kernelnya. Ini menjadi lebih penting setelah kedatangan platform containerisasi sumber terbuka Docker pada tahun 2013. (Docker sangat populer sehingga frasa "kontainer Docker" dan kata "kontainer" sering digunakan secara bergantian). Saat ini, kontainer kompatibel dengan banyak sistem operasi lain selain Linux, termasuk Windows [2].

C Hasil dan Pembahasan

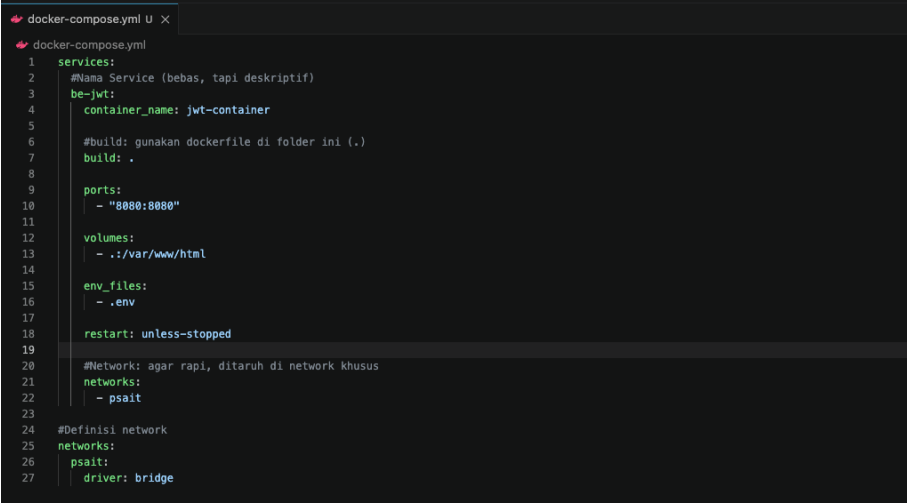
Pada pertemuan ini, mahasiswa melanjutkan praktikum sebelumnya yaitu terkait dengan membuat container dengan Dockerfile. Pada pertemuan ini, mahasiswa akan mempelajari bagaimana menjalankan container dengan menggunakan Docker Compose.

C.1 Hapus container pada pertemuan sebelumnya

Sebelum memulai praktikum, mahasiswa diminta untuk menghapus container yang telah dibuat pada pertemuan sebelumnya. Hal ini dilakukan agar tidak terjadi konflik antara container yang lama dengan container yang akan dibuat pada pertemuan ini.

C.2 Membuat file docker-compose.yml

Buat file docker-compose.yml di root projek. Ada dua bagian utama dalam file docker-compose.yml, yaitu bagian services dan networks.



```
1 services:
2   #Nama Service (bebas, tapi deskriptif)
3   be-jwt:
4     container_name: jwt-container
5
6     #build: gunakan dockerfile di folder ini (.)
7     build: .
8
9     ports:
10      - "8080:8080"
11
12     volumes:
13      - ../var/www/html
14
15     env_files:
16      - .env
17
18     restart: unless-stopped
19
20     #Network: agar rapi, ditaruh di network khusus
21     networks:
22      - psalt
23
24 #Definisi network
25 networks:
26   psalt:
27     driver: bridge
```

Gambar 1: Contoh file docker-compose.yml

C.3 Menjalankan container dengan Docker Compose

Setelah membuat file docker-compose.yml, mahasiswa dapat menjalankan command docker-compose up -d untuk menjalankan container secara detached (background) dan melihat perubahan yang terjadi pada container.

```
praktikan@psait:~/psait/php-jwt-example$ docker compose up -d
=> reading from stdin 5428
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 476B
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/8] FROM docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd
=> resolve docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd
=> [internal] load build context
=> transferring context: 280.22kB
=> CACHED [2/8] RUN docker-php-ext-install pdo_mysql
=> CACHED [3/8] RUN a2enmod rewrite headers
=> CACHED [4/8] RUN sed -ri 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf
=> CACHED [5/8] RUN rm -f /var/www/html/index.html
=> CACHED [6/8] RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf && sed -ri 's/:80/>:8080/g' /etc/apache2/sites-available/000-default.conf
=> CACHED [7/8] WORKDIR /var/www/html
=> [8/8] COPY . /var/www/html
=> exporting to image
=> exporting layers
=> exporting manifest sha256:4067898c18ad03a66536acf77ef92cb6c8705c3a00e765d6910e5dcf67f75368
=> exporting config sha256:c0435cda19a2dfb7bc47992d58fd591361b32da2b3f6042c90835a6f366ed1008
=> exporting attestation manifest sha256:d28d713345861f73473f4f6d8e6591d0b1e5fb109c35e3645b3247b580c0ed7b
=> exporting manifest list sha256:a2f447a1506fd77a35459ece13743aa77d92fcb3db89e4a54a43f0074cdc4088
=> naming to docker.io/library/php-jwt-example-be-jwt:latest
=> unpacking to docker.io/library/php-jwt-example-be-jwt:latest
=> resolving provenance for metadata file
[*] up 3/3
✔ Image php-jwt-example-be-jwt Built
✔ Network php-jwt-example_psait Created
✔ Container jwt-container Started
```

Gambar 2: Menjalankan container dengan Docker Compose

C.4 Mengecek status docker compose

Setelah menjalankan command docker-compose up -d, mahasiswa dapat mengecek status container dengan menggunakan command

```
1 docker-compose ps -a
```

Command ini akan menampilkan daftar container yang sedang berjalan beserta statusnya.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
jwt-container	php-jwt-example-be-jwt	"docker-php-entrypoi..."	be-jwt	About a minute ago	Up About a minute	80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

Gambar 3: Mengecek status docker compose

C.5 Build image ulang salah satu service docker compose

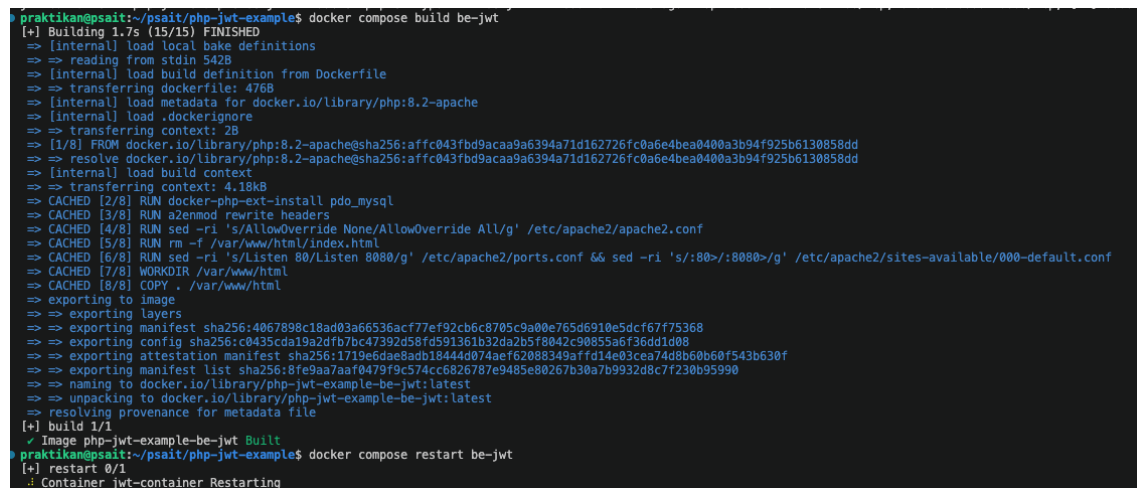
rebuild salah satu service pada docker compose dapat dilakukan dengan menggunakan command

```
1 docker-compose build nama\_service
```

Command ini akan membangun ulang image untuk service yang telah ditentukan.

Setelah itu, restart service dengan command

```
1 docker-compose restart nama\_service
```



```
praktikan@psait:~/psait/php-jwt-example$ docker compose build be-jwt
[+] Building 1.7s (15/15) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 542B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 476B
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/8] FROM docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd
=> => resolve docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd
=> [internal] load build context
=> => transferring context: 4.18kB
=> CACHED [2/8] RUN docker-php-ext-install pdo_mysql
=> CACHED [3/8] RUN a2enmod rewrite headers
=> CACHED [4/8] RUN sed -ri 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf
=> CACHED [5/8] RUN rm -f /var/www/html/index.html
=> CACHED [6/8] RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf && sed -ri 's/:80>/:8080>/g' /etc/apache2/sites-available/000-default.conf
=> CACHED [7/8] WORKDIR /var/www/html
=> CACHED [8/8] COPY . /var/www/html
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:4067898c18ad03a66536acf77ef92cb6c8705c9a00e765d6910e5dcf67f75368
=> => exporting config sha256:c0435cd19a2dfb7bc47392d58fd591361b32da2b5f8042c90855a6f36dd1d08
=> => exporting attestation manifest sha256:1719e6dae8adb18444d074aef62088349affd14e03cea74d8b60b60f543b630f
=> => exporting manifest list sha256:8fe9aa7aaf0479f9c574cc6826787e9485e80267b30a7b9932d8c7f230b95990
=> => naming to docker.io/library/php-jwt-example-be-jwt:latest
=> => unpacking to docker.io/library/php-jwt-example-be-jwt:latest
=> => resolving provenance for metadata file
[+] build 1/1
✔ Image php-jwt-example-be-jwt Built
praktikan@psait:~/psait/php-jwt-example$ docker compose restart be-jwt
[+] restart 0/1
.: Container jwt-container Restarting
```

Gambar 4: Build ulang salah satu service docker compose

C.6 Menambahkan service baru pada docker compose

Menambahkan service baru yaitu database_mysql pada docker compose dapat dilakukan dengan menambahkan konfigurasi service baru pada file docker-compose.yml.

```

1  services:
3  be-jwt:
20      #Network: agar rapi, ditaruh di network khusus
21      networks:
22      - psait
23
24  database_mysql:
25      image: mysql:8.0
26      container_name: db_mysql_jwt
27      restart: unless-stopped
28      networks:
29      - psait
30      volumes:
31      - db_data:/var/lib/mysql
32      - ./php_jwt_db.sql:/docker-entrypoint-initdb.d/init.sql
33      environment:
34      MYSQL_DATABASES: ${DB_NAME}
35      MYSQL_USER: ${DB_USER}
36      MYSQL_PASSWORD: ${DB_PASSWORD}
37      MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
38      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""}
39

```

Gambar 5: Menambahkan service baru pada docker compose

Setelah itu, tambahkan persistent volume untuk menyimpan data database agar tidak hilang saat container direstart.

```

58
59  volumes:
60  |  db_data:
61
62  #Definisi network
63  networks:
64  |  psait:
65  |  |  driver: bridge
66
67

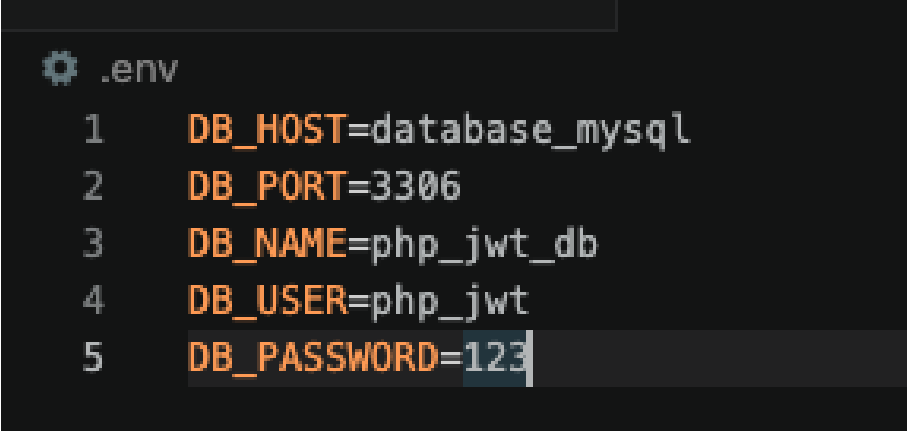
```

Gambar 6: Menambahkan persistent volume pada docker compose

Sekarang, karena service be-jwt akan bergantung pada database_mysql, maka tambahkan konfigurasi depends_on pada service be-jwt.

```
1  be-jwt :  
2      depends\_on :  
3          - database_mysql
```

Service be-jwt tidak begitu saja akan menggunakan service database_mysql yang telah dibuat. Praktikan perlu mengubah file .env dan mengarahkan ke service database_mysql. Di file .env, ubah DB_HOST menjadi nama service database yang telah dibuat seperti pada gambar Konfigurasi DB_HOST database_mysql ini akan ditranslate oleh docker sendiri. Namun, praktikan tetap bisa mengetahui IP Address nya dengan masuk ke sistem Docker. Berikut adalah contoh konfigurasi pada file .env



```
1  DB_HOST=database_mysql  
2  DB_PORT=3306  
3  DB_NAME=php_jwt_db  
4  DB_USER=php_jwt  
5  DB_PASSWORD=123
```

Gambar 7: Mengubah konfigurasi DB_HOST pada file .env

Secara keseluruhan, pembuatan docker-compose.yml dengan menambahkan service database_mysql dapat dilihat pada gambar 8.


```

1  services:
2    #Nama Service (bebas, tapi deskriptif)
3    be-jwt:
4      container_name: jwt-container
5      #build: gunakan dockerfile di folder ini (.)
6      build: .
7      ports:
8        - "8080:8080"
9      volumes:
10       - ./var/www/html
11      env_file:
12       - .env
13      restart: unless-stopped
14
15      #Network: agar rapi, ditaruh di network khusus
16      networks:
17       - psait
18
19      depends_on:
20       - database_mysql
21
22  database_mysql:
23    image: mysql:8.0
24    container_name: db_mysql_jwt
25    restart: unless-stopped
26    networks:
27     - psait
28    volumes:
29     - db_data:/var/lib/mysql
30     - ./php_jwt_db.sql:/docker-entrypoint-initdb.d/init.sql
31
32    environment:
33     MYSQL_DATABASE: ${DB_NAME}
34     MYSQL_USER: ${DB_USER}
35     MYSQL_PASSWORD: ${DB_PASSWORD}
36     MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
37     MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""}
38
39  volumes:
40    db_data:
41
42  #Definisi network
43  networks:
44    psait:
45     driver: bridge
46

```

Gambar 8: File docker-compose.yml dengan service database_mysql

Untuk menerapkan semua perubahan yang telah dilakukan, jalankan perintah

```
1  docker-compose up -d
```

Dapat dilihat pada gambar 9 bahwa semua service telah berjalan dengan baik, termasuk service database_mysql yang baru ditambahkan.

```
praktikan@psait:~/psait/php-jwt-example$ docker compose up -d
[+] up 17/17
✓ Image mysql:8.0 Pulled
✓ Volume php-jwt-example_db_data Created
✓ Container db_mysql_jwt Started
✓ Container jwt-container Started
praktikan@psait:~/psait/php-jwt-example$
```

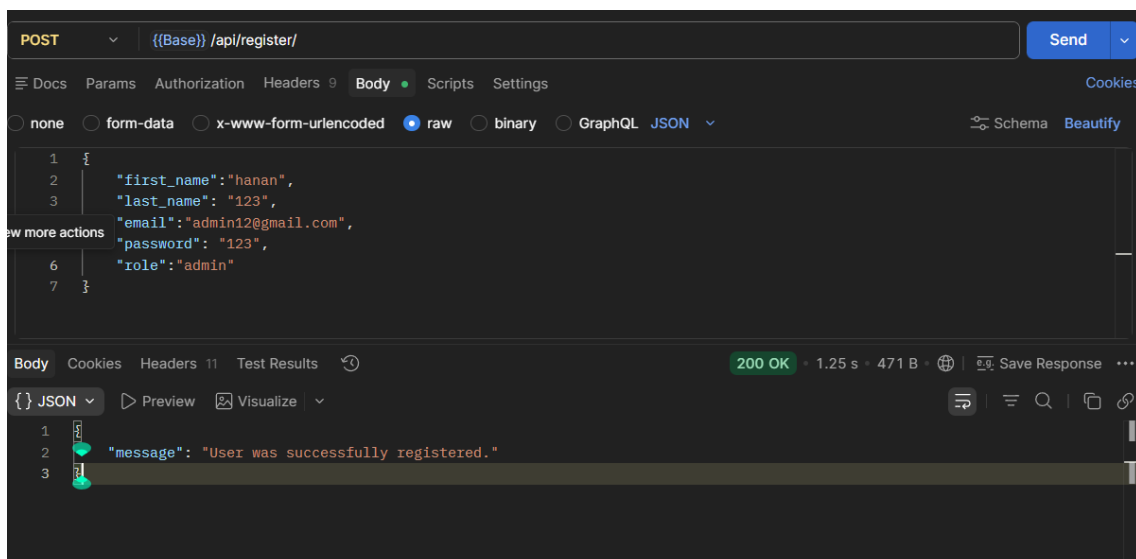
Gambar 9: Menjalankan docker compose dengan service database_mysql

C.7 Mengecek api di Postman

Setelah semua service berjalan dengan baik, mahasiswa dapat mengecek API dengan database dari Docker. Pastikan untuk mengubah URL endpoint API sesuai dengan port yang telah ditentukan pada file docker-compose.yml. Berikut url dari BASE.

```
1 http://10.33.35.71:8080/api/
```

Seperti biasa, jalankan tiga endpoint untuk mengecek apakah API sudah berjalan dengan baik, yaitu endpoint untuk register, login, serta menampilkan semua data user. Berikut adalah contoh hasil pengecekan API di Postman.



Gambar 10: Pengecekan endpoint register API di Postman

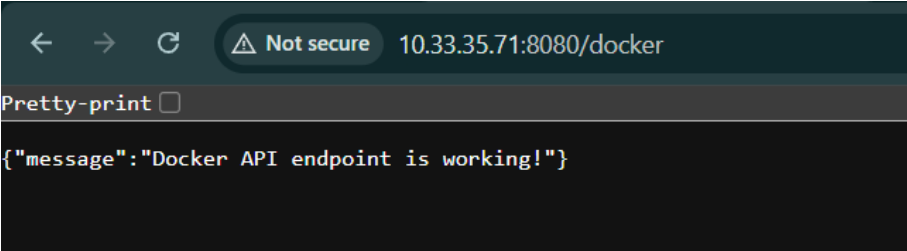

```

index.php
15 switch ($requestUri) {
16     case '/api/register':
17         if (file_exists('api/register/index.php')) {
18             require 'api/register/index.php';
19         } else {
20             http_response_code(500);
21             echo json_encode(["message" => "File api/register/index.php tidak ditemukan!"]);
22         }
23         break;
24
25     case '/api/users':
26         if (file_exists('api/users/index.php')) {
27             require 'api/users/index.php';
28         } else {
29             http_response_code(500);
30             echo json_encode(["message" => "File api/users/index.php tidak ditemukan!"]);
31         }
32         break;
33
34     case '/docker':
35         http_response_code(200);
36         echo json_encode(["message" => "Docker API endpoint is working!"]);
37         break;
38
39     case '/':
40     case '/index.php':
41         http_response_code(200);
42         echo json_encode(array("message" => "Welcome to the API service"));
43         break;
44
45     default:
46         http_response_code(404);
47         echo json_encode(array(
48             "message" => "Endpoint not found.",
49             "debug_uri" => $requestUri
50         ));
51         break;
52
53     --

```

Gambar 13: Menambahkan endpoint /docker pada file index.php

Setelah melakukan perubahan, simpan file tersebut dan cek kembali langsung di browser dengan url <http://10.33.35.71:8080/docker>. Akan muncul respons seperti pada gambar 14 yang menandakan bahwa perubahan kode telah langsung terlihat tanpa perlu melakukan rebuild pada container.



The screenshot shows a web browser window with the address bar displaying "10.33.35.71:8080/docker". The browser indicates the connection is "Not secure". Below the address bar, there is a "Pretty-print" checkbox. The main content area of the browser displays a JSON response: `{"message": "Docker API endpoint is working!"}`.

Gambar 14: Hasil pengecekan endpoint /docker setelah melakukan perubahan kode

D Tugas dan Analisis

1. Tambahkan service phpMyAdmin ke dalam orkestrasi Docker Compose! Forward port phpMyAdmin tersebut ke port selain port 8080 agar tidak tabrakan. Untuk menambahkan service phpMyAdmin, tambahkan konfigurasi service baru pada file docker-compose.yml seperti pada gambar 15. Pastikan untuk mengubah port forwarding selain 8080 (mahasiswa menggunakan port 8081) agar tidak tabrakan dengan service be-jwt yang telah dibuat sebelumnya.

```
mysql_root_password: ${DB_ROOT_PASS:-""}

phpmyadmin:
  image: phpmyadmin:latest
  container_name: phpmyadmin_jwt
  restart: unless-stopped

  ports:
    - "8081:80"

  environment:
    PMA_HOST: database_mysql
    PMA_PORT: 3306
    MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""}

  depends_on:
    - database_mysql

  networks:
    - psait
```

Gambar 15: Menambahkan service phpMyAdmin pada docker compose

Berdasarkan gambar 15, mahasiswa mendefinisikan sebuah *service* bernama `phpmyadmin` yang berfungsi sebagai antarmuka grafis berbasis web untuk mengelola basis data MySQL secara mudah, tanpa harus menggunakan baris perintah atau terminal.

Secara lebih terperinci, konfigurasi tersebut diawali dengan penggunaan citra resmi versi terbaru dari Docker Hub yang dideklarasikan melalui sintaks `image:`

`phpmyadmin:latest`. Kontainer ini kemudian diberi identitas khusus berupa nama `phpmyadmin_jwt` melalui parameter `container.name`. Agar layanan dapat diakses oleh pengguna dari luar kontainer, terdapat pengaturan `ports`:
- `"8081:80"` yang berfungsi memetakan port internal kontainer (port 80) ke port komputer lokal mahasiswa (port 8081). Dengan demikian, mahasiswa dapat mengakses antarmuka phpMyAdmin secara langsung melalui peramban web dengan alamat `http://localhost:8081`.

Selanjutnya, terdapat blok `environment` yang memuat variabel lingkungan esensial agar phpMyAdmin dapat terhubung dengan kontainer basis data. Melalui parameter `PMA_HOST`, konfigurasi ini diarahkan untuk mengenali kontainer basis data yang bernama `database_mysql` melalui jaringan internal pada port standar 3306 (`PMA_PORT`). Sistem keamanan atau kata sandi utama juga diselaraskan menggunakan variabel `MYSQL_ROOT_PASSWORD`. Untuk memastikan kelancaran proses pemuatan sistem, direktif `depends_on` digunakan guna menjamin bahwa kontainer `database_mysql` akan dijalankan terlebih dahulu sebelum kontainer phpMyAdmin diaktifkan. Seluruh interaksi dan komunikasi antar-kontainer ini diwadahi di dalam sebuah jaringan virtual Docker yang sama, yang didefinisikan oleh mahasiswa pada bagian `networks` dengan nama `psait`.

2. Lakukan CRUD database aplikasi `php_jwt-db` dengan menggunakan phpMyAdmin yang telah dimuat di Docker Compose! Pastikan semua CRUD berjalan untuk memastikan keberhasilan penambahan service phpMyAdmin yang kalian lakukan!

(a) CREATE

Mahasiswa melakukan penambahan data user baru dengan `last_name` coba secara langsung di phpMyAdmin, akan muncul pesan berhasil seperti

pada gambar 16.



Gambar 16: Menambahkan data user baru di phpMyAdmin

(b) UPDATE

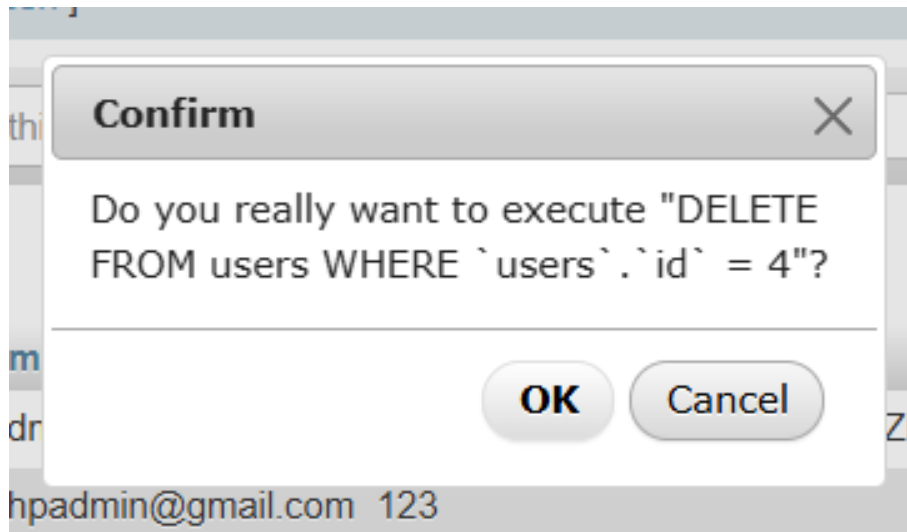
Mahasiswa melakukan perubahan data user coba dengan mengubah last_name menjadi coba edit secara langsung di phpMyAdmin



Gambar 17: Mengubah data user coba di phpMyAdmin

(c) DELETE

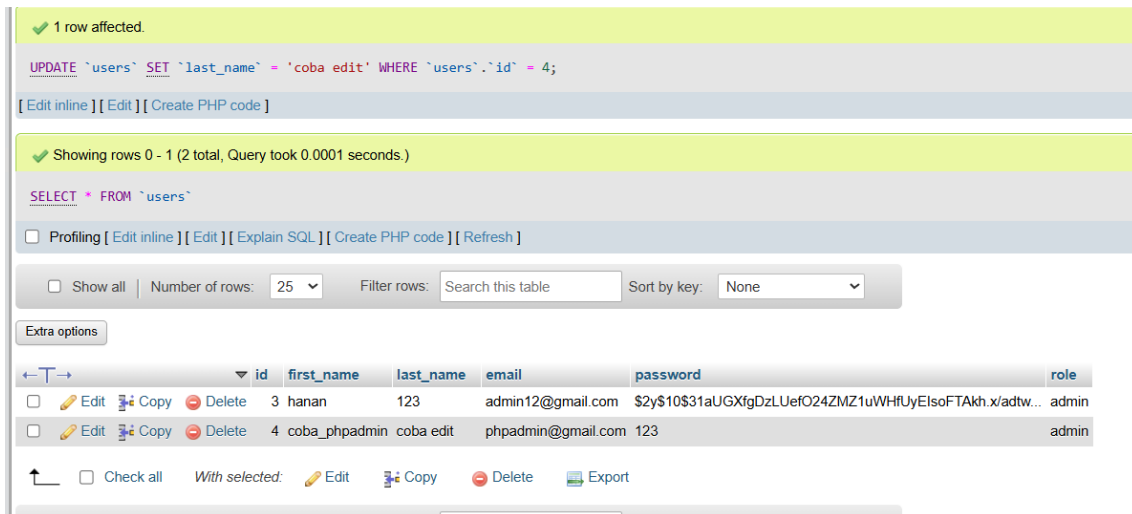
Mahasiswa melakukan penghapusan data user coba edit secara langsung di phpMyAdmin, akan muncul konfirmasi dialog seperti pada gambar 18.



Gambar 18: Menghapus data user coba edit di phpMyAdmin

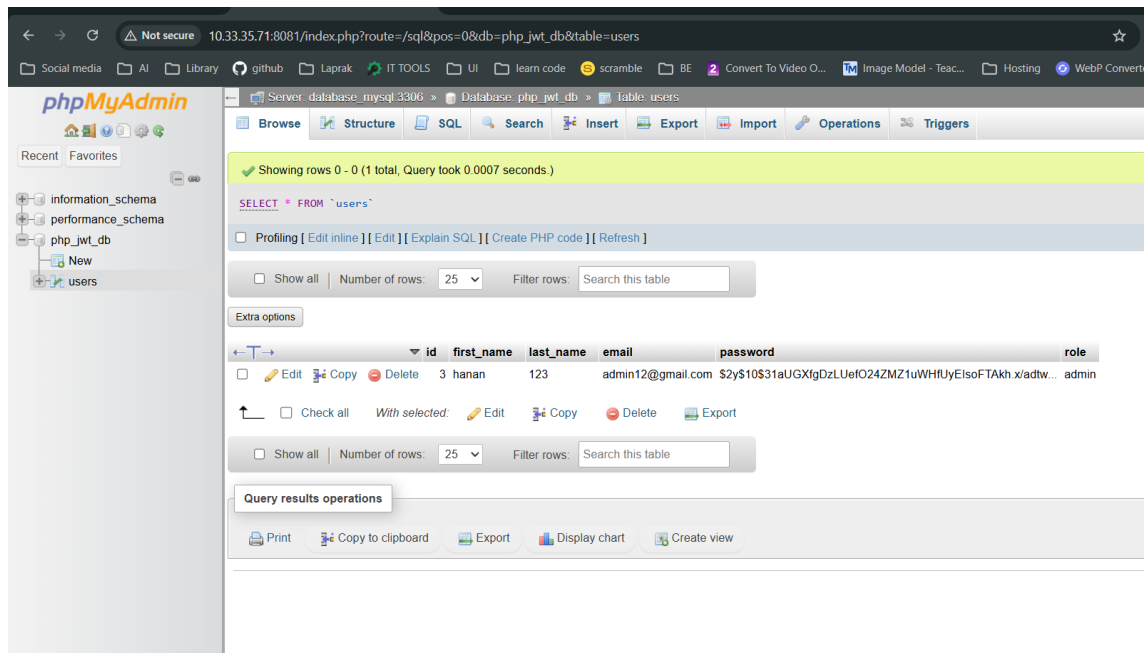
(d) READ

Mahasiswa melakukan pengecekan data user coba edit yang baru diubah
19.



Gambar 19: Mengecek data user coba yang telah diubah di phpMyAdmin

Mahasiswa juga melakukan pengecekan data user coba yang telah dihapus
seperti pada gambar 20.



Gambar 20: Mengecek data user coba yang telah dihapus di phpMyAdmin

E Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan Docker Compose sangat mempermudah proses orkestrasi dan manajemen banyak kontainer secara bersamaan apabila dibandingkan dengan mengeksekusi perintah CLI secara manual. Mahasiswa telah berhasil menyusun dan mengonfigurasi berkas `docker-compose.yml` untuk mengintegrasikan layanan aplikasi pemrograman antarmuka (API), basis data MySQL, serta antarmuka phpMyAdmin ke dalam satu jaringan virtual yang saling terhubung. Selain itu, mahasiswa juga mampu mengimplementasikan fitur *hot-reloading* dengan memanfaatkan *volumes* sehingga perubahan pada kode sumber dapat langsung diamati tanpa perlu membangun ulang (*rebuild*) citra kontainer, serta memahami penerapan manajemen variabel lingkungan menggunakan berkas `.env`.

Penerapan arsitektur tersebut terbukti berjalan dengan baik secara fungsional. Keberhasilan praktikum ini dibuktikan dengan berjalannya pengujian titik akhir (*endpoint*) API secara lancar melalui aplikasi Postman. Selain itu, integrasi antarmuka phpMyAdmin juga beroperasi sesuai harapan, yang ditunjukkan melalui keberhasilan mahasiswa dalam melakukan berbagai operasi CRUD (Create, Read, Update, Delete) pada basis data secara langsung tanpa mengalami kendala.

F Daftar Pustaka

- [1] N. Yulian, “Berkenalan dengan docker compose (docker) contoh kasus dockerize php , apache, dan postgresql,” <https://medium.com/@nanoyulian/berkenalan-dengan-docker-compose-docker-63208f45ca4c>, 2020, [Accessed 26-05-2026].
- [2] I. S. Stephanie Susnjara, “Apa yang dimaksud dengan orkestrasi container? — IBM — ibm.com,” <https://www.ibm.com/id-id/think/topics/container-orchestration>, [Accessed 26-05-2026].